REPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
****
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR
******
IAI-CAMEROUN
*****
BP 13719 (Yaoundé)
Tel :(237)22729957/22729958

REPUBLIC OF CAMEROON
PEACE-WORK- FATHERLAND
*****
MINISTERY OF HIGH EDUCATION
*****
IAI-CAMEROUN
*****
BP 13719 (Yaoundé)
Tel :(237)22729957/22729958

INSTITUT AFRICAIN D'INFORMATIQUE

CONCEPTION D'UNE BASE DE DONNEE DE GESTION DE CUISINE

EXPOSE INITIATION AUX BASES DE DONNEES

EXPOSANTS:

-SAKAM KAMGANG

- SAMIRA AOUDOU

-NEGUIM MAXIM

-MINYEMMECK

EXAMINATEUR : MONSIEUR MEMBOU

Table des matières

1. INTRODUCTION GÉNÉRALE 5

1.1. CONTEXTE ET MOTIVATION 6

1.2 OBJECTIFS DU PROJET 7

1.3 PLAN DU DOCUMENT 8

2. ANALYSE DU BESOIN 9

2.1 Contexte de la restauration 9

2.2 LES ACTEURS DU SYSTEME 10

2.3 Les entités métier 10

2.4 Recueil des besoins utilisateurs 11

3. Dictionnaire des données 14

4. MODÉLISATION CONCEPTUELLE 20

5. MODELE LOGIQUE DE DONNEES 31

6.1 PROCESSUS DE NORMALISATION APPLIQUE 36

7. ÉVALUATION DES SYSTÈMES DE GESTION DE BASES DE DONNÉES POUR LE PROJET CUISINE 41

8. IMPLÉMENTATION SQL 51

8.1 R1.1 : Sélection complète 51

8.2 R1.2 : Colonnes spécifiques 51

8.3 R1.3 : Alias de colonnes 51

8.4 R1.4 : Suppression des doublons 51

8.5 R1.5 : Tri des résultats 51

8.6 R1.6 : Limitation du nombre 52

8.7 R2.1 : Égalité 52

8.8 R2.2 : Différence 52

8.9 R2.3 : Comparaison numérique 52
8.10 R2.4 : Intervalle 52
8.11 R2.5 : Liste de valeurs 52
8.12 R2.6 : Recherche par motif 53
8.13 R2.7 : Test de nullité 53
8.14 R2.8 : Conditions combinées (ET) 53
8.15 R2.9 : Conditions combinées (OU) 53
8.16 R3.1 : Jointure interne 53
8.17 R3.2 : Jointure avec alias 54
8.18 R3.3 : Jointure multiple (3 tables) 54
8.19 R3.4 : Jointure externe gauche 54
8.20 R4.1 : Comptage 54
8.21 R4.2 : Somme 55
8.22 R4.3 : Moyenne 55
8.23 R4.4 : Min/Max 55
8.24 R4.5 : Filtrage de groupes 55
8.25 R4.6 : Statistiques complètes 55
8.26 R5.1 : Mise à jour simple 56
8.27 R5.2 : Mise à jour calculée 56
8.28 R5.3 : Mise à jour multi-colonnes 56
8.29 R5.4 : Mise à jour avec jointure 56
8.30 R6.1 : Suppression ciblée 57
8.31 R6.2 : Suppression conditionnelle 57
8.32 R6.3 : Suppression avec sous-requête 57
8.33 R7.1 : Insertion complète 57
8.34 R7.2 : Insertion partielle 57
8.35 R7.3 : Insertion multiple 58
8.36 R7.4 : Insertion par copie 58
8.37 R8.1 : Sous-requête scalaire 58
8.38 R8.2 : Sous-requête corrélée 58
8.39 R8.3 : Opérateur IN 59
8.40 R8.4 : Opérateur EXISTS 59
8.41 R8.5 : Opérateur NOT EXISTS 59

8.42 R9.1 : Union sans doublon 59
8.43 R9.2 : Union avec doublons 59
8.44 R9.3 : Intersection 60
8.45 R9.4 : Différence 60
8.46 R10.1 : Numérotation 60
8.47 R10.2 : Classement avec ex-aequo 60
8.48 R10.3 : Classement par groupe 61
8.49 R10.4 : Navigation 61
8.50 R10.5 : Somme cumulative 61
8.51 R11.1 : Vue stock critique 62
8.52 R11.2 : Vue carte recettes 62
8.53 R11.3 : Vue planning 62
8.54 R11.4 : Vue compétences 62
8.55 R12.1 : Calcul rentabilité menu 62
8.56 R12.2 : Génération liste courses 63
8.57 R12.3 : Informations système 63
8.58 R13.1 : Rentabilité menus 63
8.59 R13.2 : Analyse ABC (80/20) 63
8.60 R13.3 : Matrice compétences 64
8.61 R13.4 : Recettes réalisables 64
8.62 R13.5 : Tendances RH 65

9. CONCLUSION 66

10. BIBLIOGRAPHIE 68

11. GLOSSAIRE 69

1. INTRODUCTION GÉNÉRALE

Dans le contexte actuel de la transformation numérique, la gestion efficace des informations est devenue un enjeu stratégique majeur pour les entreprises de tous secteurs. Le domaine de la restauration et de la cuisine ne fait pas exception à cette réalité. Que ce soit pour la gestion des stocks d'ingrédients, le suivi des recettes, la planification des menus ou la traçabilité des produits, une base de données bien conçue constitue le socle indispensable d'un système d'information performant.

Ce document présente la conception et la réalisation d'une base de données relationnelle dédiée à la gestion d'une cuisine professionnelle. L'objectif est de fournir une solution complète permettant de modéliser l'ensemble des entités et des relations inhérentes à ce domaine d'activité, en appliquant les principes fondamentaux de l'algèbre relationnelle et de la normalisation des données.

1.1. CONTEXTE ET MOTIVATION

Une cuisine professionnelle, qu'il s'agisse d'un restaurant, d'un hôtel ou d'une entreprise de restauration collective, gère quotidiennement un volume considérable d'informations :

- Des ingrédients avec leurs caractéristiques (nom, catégorie, unité de mesure, prix, fournisseur)
- Des recettes composées d'ingrédients en quantités spécifiques
- Des menus regroupant plusieurs recettes
- Des chefs et cuisiniers responsables des préparations
- Des commandes passées auprès des fournisseurs
- Des stocks nécessitant un suivi rigoureux

La gestion manuelle de ces informations s'avère rapidement inefficace et source d'erreurs. Une base de données informatisée permet d'automatiser ces tâches, d'assurer la cohérence des données et de faciliter l'extraction d'informations stratégiques

1.2 OBJECTIFS DU PROJET

Ce projet de conception de base de données vise à atteindre les objectifs suivants :

- Modéliser de manière exhaustive les entités du domaine culinaire (ingrédients, recettes, menus, chefs, fournisseurs)
- Identifier et formaliser les relations entre ces entités
- Appliquer la méthode Merise pour la conception conceptuelle et logique
- Utiliser l'algèbre relationnelle pour exprimer les requêtes utilisateurs
- Normaliser le schéma relationnel pour éliminer les redondances
- Produire le code SQL de création et d'interrogation de la base

1.3 PLAN DU DOCUMENT

Ce document présente d'abord le contexte et les objectifs du projet, puis l'analyse des besoins du domaine étudié. Il décrit ensuite les données à travers le dictionnaire et leur modélisation, avant de proposer un modèle logique et une normalisation adaptée.

L'implémentation de la base de données est réalisée en SQL à travers différentes requêtes d'exploitation. Une évaluation des systèmes de gestion de bases de données est également effectuée afin de justifier le choix technologique.

Le document se conclut par une synthèse du travail, accompagnée des références bibliographiques et d'un glossaire.

2. ANALYSE DU BESOIN

Ce chapitre présente l'analyse des besoins pour la conception de la base de données d'une cuisine. Nous décrivons le domaine d'application, recensons les données nécessaires et établissons le dictionnaire des données.

Présentation du domaine de la cuisine

2.1 Contexte de la restauration

La cuisine professionnelle englobe un large spectre d'activités allant du restaurant traditionnel à la restauration collective, en passant par la restauration rapide et les services de traiteur. Quelle que soit la forme, la gestion efficace des ressources culinaires est un facteur clé de réussite.

Dans une cuisine professionnelle, plusieurs activités coexistent et nécessitent une coordination optimale :

- L'approvisionnement : gestion des fournisseurs et des commandes
- Le stockage : suivi des inventaires et gestion des DLC/DLUO
- La préparation : exécution des recettes selon les fiches techniques
- La planification : élaboration des menus et organisation des services
- Le contrôle qualité : traçabilité et respect des normes d'hygiène

Gestion des DLC (Date Limite de Consommation) : C'est une limite impérative. Une fois cette date dépassée, le produit peut présenter un risque pour la santé et sa vente est **interdite**.
DLUO (Date Limite d'Utilisation Optimale) : le produit reste consommable après cette date sans risque pour la santé. Cependant, il peut perdre ses qualités : moins de goût, texture différente (biscuits moins croquants), ou perte de vitamines.

2.2 LES ACTEURS DU SYSTEME

Différents acteurs interviennent dans le fonctionnement d'une cuisine professionnelle :

Le chef de cuisine : Responsable de l'ensemble de la cuisine, il définit les menus, supervise les équipes et contrôle la qualité des préparations. Il a besoin d'accéder rapidement aux recettes, aux stocks disponibles et aux informations sur les fournisseurs.

Les chefs de partie : Spécialisés dans un domaine particulier (sauces, grillades, pâtisserie...), ils consultent les recettes de leur domaine et signalent les besoins en ingrédients.

Le responsable des achats : Il gère les relations avec les fournisseurs, passe les commandes et négocie les prix. Il a besoin d'informations détaillées sur les produits et l'historique des achats.

Le gestionnaire de stocks : Il assure le suivi des inventaires, contrôle les dates de péremption et optimise les niveaux de stock.

2.3 Les entités métier

L'analyse du domaine culinaire permet d'identifier les entités fondamentales suivantes :

Les ingrédients : Ce sont les matières premières utilisées dans la préparation des plats. Chaque ingrédient possède des caractéristiques spécifiques : nom, catégorie (viandes, légumes, épices...), unité de mesure (kg, litre, unité...), prix unitaire, fournisseur.

Les recettes : Ce sont les modes opératoires détaillant la préparation d'un plat. Une recette comprend un nom, une catégorie (entrée, plat, dessert...), un temps de préparation, un nombre de couverts, et la liste des ingrédients avec leurs quantités.

Les menus : Ce sont des ensembles de recettes proposés pour un service ou un événement. Un menu est caractérisé par un nom, une date, un type (déjeuner, dîner, buffet...) et la liste des plats qui le composent.

Les chefs : Ce sont les professionnels responsables de la préparation des plats. Chaque chef possède un nom, un prénom, une spécialité, un niveau de compétence et une date d'embauche.

Les fournisseurs : Ce sont les entreprises fournissant les ingrédients. Un fournisseur est décrit par sa raison sociale, son adresse, son téléphone, son email et sa spécialité.

2.4 Recueil des besoins utilisateurs

2.4.1 Méthodologie de recueil

Le recueil des besoins est une étape cruciale qui détermine la qualité de la future base de données. Plusieurs techniques ont été utilisées :

a. Les entretiens avec les utilisateurs (chefs, responsables, gestionnaires)

b. L'analyse des documents existants (fiches techniques, bons de commande, inventaires)

c. L'observation des processus métier en situation réelle

d. L'étude des applications existantes et de leurs limites

2.4.2 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit permettre de faire. Pour notre base de données cuisine, les besoins fonctionnels principaux sont :

a. Gestion des ingrédients : Le système doit permettre de créer, modifier, supprimer et consulter les informations sur les ingrédients. Il doit également permettre de rechercher des ingrédients par catégorie, par fournisseur ou par plage de prix.

b. Gestion des recettes : Le système doit permettre de gérer les fiches techniques des recettes, incluant les ingrédients nécessaires et leurs quantités. Il doit également permettre de calculer le coût de revient d'une recette.

c. Gestion des menus : Le système doit permettre de composer des menus en sélectionnant des recettes existantes et de planifier les services.

d. Gestion des stocks : Le système doit permettre de suivre les niveaux de stock, de gérer les entrées et sorties, et de générer des alertes pour les réapprovisionnements.

e. Gestion des fournisseurs : Le système doit permettre de gérer le référentiel fournisseurs et de tracer l'historique des commandes.

2.4.3 Besoins non fonctionnels

Les besoins non fonctionnels décrivent les contraintes et qualités attendues du système :

- Performance : Les requêtes doivent s'exécuter en moins de 2 secondes
- Disponibilité : Le système doit être accessible 12 heures par jour, 6 jours sur 7
- Sécurité : Les accès doivent être contrôlés par authentification

- Ergonomie : L'interface doit être intuitive et facile d'utilisation
- Évolutivité : Le système doit pouvoir accueillir de nouvelles fonctionnalités

2.4.4 Règles de gestion

Les règles de gestion définissent les contraintes métier qui doivent être respectées par le système :

i. RG1 : Chaque ingrédient possède un code unique et appartient à une seule catégorie.

ii. RG2 : Une recette est composée d'au moins un ingrédient et peut être utilisée dans plusieurs menus.

iii. RG3 : Un menu contient au moins une recette et est associé à une date de service.

iv. RG4 : Un chef peut être responsable de plusieurs recettes selon sa spécialité.

v. RG5 : Un fournisseur peut fournir plusieurs ingrédients, mais chaque ingrédient a un fournisseur principal.

vi. RG6 : La quantité en stock d'un ingrédient ne peut pas être négative.

3. Dictionnaire des données

3.1 Principe du dictionnaire

Le dictionnaire des données est un document de référence qui recense et décrit de manière exhaustive toutes les données nécessaires au système d'information. Chaque donnée est caractérisée par :

- Un code mnémonique (nom interne)
- Un libellé explicite
- Un type (alphanumérique, numérique, date...)
- Un format (longueur maximale)
- Une catégorie (identifiant ou propriété)

DONNEES DE L'ENTITE INGREDIENT

Code	Libellé	Type	Format	Catégorie
CODE_ING	Code ingrédient	Numérique	5	Identifiant
NOM_ING	Nom de l'ingrédient	Alphanumérique	50	Propriété
CAT_ING	Catégorie	Alphanumérique	30	Propriété
UNITE	Unité de mesure	Alphanumérique	10	Propriété
PRIX_UNIT	Prix unitaire	Numérique décimal	10,2	Propriété
QTE_STOCK	Quantité en stock	Numérique décimal	10,2	Propriété
SEUIL_MIN	Seuil minimum	Numérique décimal	10,2	Propriété

DONNEES DE L'ENTITE RECETTE

Code	Libellé	Type	Format	Catégorie
CODE_REC	Code recette	Numérique	5	Identifiant
NOM_REC	Nom de la recette	Alphanumérique	50	Propriété
CAT_REC	Catégorie	Alphanumérique	30	Propriété
TEMPS_PREP	Temps de préparation	Numérique	3	Propriété
NB_COUVERTS	Nombre de couverts	Numérique	3	Propriété
DIFFICULTE	Niveau de difficulté	Alphanumérique	10	Propriété

DONNEES DE L'ENTITE MENU

Code	Libellé	Type	Format	Catégorie
CODE_MENU	Code menu	Numérique	5	Identifiant
NOM_MENU	Nom du menu	Alphanumérique	50	Propriété
DATE_MENU	Date du service	Date	10	Propriété
TYPE_MENU	Type de menu	Alphanumérique	20	Propriété
PRIX_MENU	Prix du menu	Numérique décimal	10,2	Propriété

Données de l'entité CHEF

Code	Libellé	Type	Format	Catégorie
CODE_CHEF	Code chef	Numérique	5	Identifiant
NOM_CHEF	Nom du chef	Alphanumérique	30	Propriété
PRENOM_CHEF	Prénom du chef	Alphanumérique	30	Propriété
SPECIALITE	Spécialité	Alphanumérique	30	Propriété
NIVEAU	Niveau de compétence	Alphanumérique	15	Propriété
DATE_EMB	Date d'embauche	Date	10	Propriété

Données de l'entité FOURNISSEUR

Code	Libellé	Type	Format	Catégorie
CODE_FOUR	Code fournisseur	Numérique	5	Identifiant
RAISON_SOC	Raison sociale	Alphanumérique	50	Propriété
ADRESSE	Adresse	Alphanumérique	100	Propriété
TELEPHONE	Téléphone	Alphanumérique	15	Propriété
EMAIL	Adresse email	Alphanumérique	50	Propriété
SPECIALITE	Spécialité	Alphanumérique	30	Propriété

DONNEES DES ASSOCIATIONS

Une association représente un lien sémantique entre plusieurs entités. Elle peut être caractérisée par des propriétés propres. La cardinalité d'une association indique le nombre minimum et maximum d'occurrences d'une entité pouvant participer à l'association.

Les cardinalités s'expriment sous la forme (min, max) où :

- min = 0 signifie une participation optionnelle
- min = 1 signifie une participation obligatoire
- max = 1 signifie qu'une occurrence ne peut participer qu'une seule fois
- max = n signifie qu'une occurrence peut participer plusieurs fois

L’analyse des différentes entités permettes de définir 4 associations :

1. ''COMPOSE'' qui Permet de décrire la fabrication d'une recette.
2. ''CONSTITUE'' qui permet de structurer le menu
3. ''REALISATION'' qui permet de gérer les compétences des chefs.
4. ''APPARTIENT'' qui permet de gérer l'approvisionnement.

Les associations entre entités nécessitent également des données spécifiques :

Association COMPOSE (entre RECETTE et INGREDIENT) :

Code	Libellé	Type	Format	Catégorie
CODE_REC	Code recette	Numérique	5	Identifiant
CODE_ING	Code ingrédient	Numérique	5	Identifiant
QUANTITE	Quantité nécessaire	Numérique décimal	8,2	Propriété

Association CONSTITUE (entre MENU et RECETTE) :

Code	Libellé	Type	Format	Catégorie
CODE_MENU	Code menu	Numérique	5	Identifiant
CODE_REC	Code recette	Numérique	5	Identifiant
ORDRE	Ordre de service	Numérique	2	Propriété

Association APPARTIENT (entre INGREDIENT et FOURNISSEUR) :

Code	Libellé	Type	Format	Catégorie
CODE_ING	Code ingrédient	Numérique	5	Identifiant
CODE_FOUR	Code fournisseur	Numérique	5	Identifiant

Cette étape garantit d'avoir toutes les informations métiers ainsi que leurs différentes associations ; nous allons ainsi pouvoir réaliser la modélisation conceptuelle (MCD), qui permettra de représenter de manière claire, structurée et indépendante de toute contrainte technique l'ensemble des données et des relations du système, a partir des matrices de dépendance fonctionnelles.

4. MODÉLISATION CONCEPTUELLE

4.1 Matrice des Dépendances Fonctionnelles

La dépendance fonctionnelle (DF) est un concept fondamental de la théorie des bases de données relationnelles. Deux attributs A et B sont en dépendance fonctionnelle si la connaissance de la valeur de A permet de déterminer une et une seule valeur de B.

On note A → B (A détermine B). L'attribut A est appelé source de la dépendance fonctionnelle, et B est appelé cible.

Exemple : Dans notre dictionnaire, CODE_ING → NOM_ING car la connaissance du code d'un ingrédient détermine son nom de manière unique.

Étape de Construction de la matrice

La construction des différentes matrices de dépendance fonctionnelles pour notre cuisine a suivi plusieurs étapes :

- Étape 1 : Créer le squelette de la matrice avec tous les attributs en en-têtes de lignes et de colonnes.
- Étape 2 : Marquer la propriété de réflexivité (chaque attribut détermine lui-même) avec une barre sur la diagonale.
- Étape 3 : Identifier les dépendances fonctionnelles en se posant pour chaque attribut source la question : « La connaissance d'une valeur de cet attribut permet-elle de déterminer une et une seule valeur de l'attribut cible ? »

- ✪ Étape 4 : Simplifier la matrice en éliminant les colonnes des attributs qui ne sont pas sources de dépendances fonctionnelles.

- ✪ Étape 5 : Vérifier la colonne Total pour détecter les anomalies (valeur à zéro pour un attribut non-identifiant, valeur supérieure à 1 indiquant une transitivité).

MDF pour l'entité INGREDIENT

Source \ Cible	CODE_ING	NOM_ING	CAT_ING	UNITE	PRIX_UNIT	QTE_STOCK	SEUIL_MIN	Total
CODE_ING	—	1	1	1	1	1	1	**6**
NOM_ING	0	—	0	0	0	0	0	0
CAT_ING	0	0	—	0	0	0	0	0
UNITE	0	0	0	—	0	0	0	0
PRIX_UNIT	0	0	0	0	—	0	0	0
QTE_STOCK	0	0	0	0	0	—	0	0
SEUIL_MIN	0	0	0	0	0	0	—	0

Pour l'entité INGREDIENT, les dépendances fonctionnelles sont les suivantes :

- CODE_ING → NOM_ING (le code détermine le nom)

- CODE_ING → CAT_ING (le code détermine la catégorie)

- CODE_ING → UNITE (le code détermine l'unité)
- CODE_ING → PRIX_UNIT (le code détermine le prix)
- CODE_ING → QTE_STOCK (le code détermine la quantité en stock)
- CODE_ING → SEUIL_MIN (le code détermine le seuil minimum)

CODE_ING est l'identifiant de l'entité INGREDIENT. Il détermine tous les autres attributs de l'entité.

MDF pour l'entité RECETTE

Source \ Cible	CODE_REC	NOM_REC	CAT_REC	TEMPS_PREP	NB_COUVERTS	DIFFICULTE	Total
CODE_REC	—	1	1	1	1	1	**5**
NOM_REC	0	—	0	0	0	0	0
CAT_REC	0	0	—	0	0	0	0
TEMPS_PREP	0	0	0	—	0	0	0
NB_COUVERTS	0	0	0	0	—	0	0
DIFFICULTE	0	0	0	0	0	—	0

Pour l'entité RECETTE, les dépendances fonctionnelles sont :

- CODE_REC → NOM_REC (le code détermine le nom)
- CODE_REC → CAT_REC (le code détermine la catégorie)
- CODE_REC → TEMPS_PREP (le code détermine le temps de préparation)

- CODE_REC → NB_COUVERTS (le code détermine le nombre de couverts)
- CODE_REC → DIFFICULTE (le code détermine le niveau de difficulté)

CODE_REC est l'identifiant de l'entité RECETTE.

MDF POUR MENU

Source \ Cible	CODE_MENU	NOM_MENU	DATE_MENU	TYPE_MENU	PRIX_MENU	Total
CODE_MENU	—	1	1	1	1	4
NOM_MENU	0	—	0	0	0	0
DATE_MENU	0	0	—	0	0	0
TYPE_MENU	0	0	0	—	0	0
PRIX_MENU	0	0	0	0	—	0

Pour l'entité MENU, les dépendances fonctionnelles sont les suivantes :

CODE_MENU → NOM_MENU (le code détermine le nom du menu)

CODE_MENU → DATE_MENU (le code détermine la date du service)

CODE_MENU → TYPE_MENU (le code détermine le type de menu)

CODE_MENU → PRIX_MENU (le code détermine le prix du menu)

CODE_MENU est l'identifiant de l'entité MENU. Il détermine tous les autres attributs de l'entité.

MDF pour chef

Source \ Cible	CODE_CHEF	NOM_CHEF	PRENOM_CHEF	SPECIALITE	NIVEAU	DATE_EMB	Total
CODE_CHEF	—	1	1	1	1	1	**5**
NOM_CHEF	0	—	0	0	0	0	0
PRENOM_CHEF	0	0	—	0	0	0	0
SPECIALITE	0	0	0	—	0	0	0
NIVEAU	0	0	0	0	—	0	0
DATE_EMB	0	0	0	0	0	—	0

Pour l'entité CHEF, les dépendances fonctionnelles sont les suivantes :

CODE_CHEF → NOM_CHEF (le code détermine le nom du chef)

CODE_CHEF → PRENOM_CHEF (le code détermine le prénom du chef)

CODE_CHEF → SPECIALITE (le code détermine la spécialité)

CODE_CHEF → NIVEAU (le code détermine le niveau de compétence)

CODE_CHEF → DATE_EMB (le code détermine la date d'embauche)

CODE_CHEF est l'identifiant de l'entité CHEF. Il détermine tous les autres attributs de l'entité.

MDF fournisseur

Source \ Cible	CODE_FOUR	RAISON_SOC	ADRESSE	TELEPHONE	EMAIL	SPECIALITE	Total
CODE_FOUR	—	1	1	1	1	1	**5**
RAISON_SOC	0	—	0	0	0	0	0
ADRESSE	0	0	—	0	0	0	0
TELEPHONE	0	0	0	—	0	0	0
EMAIL	0	0	0	0	—	0	0
SPECIALITE	0	0	0	0	0	—	0

Pour l'entité FOURNISSEUR, les dépendances fonctionnelles sont les suivantes :

CODE_FOUR → RAISON_SOC (le code détermine la raison sociale)

CODE_FOUR → ADRESSE (le code détermine l'adresse)

CODE_FOUR → TELEPHONE (le code détermine le téléphone)

CODE_FOUR → EMAIL (le code détermine l'adresse email)

CODE_FOUR → SPECIALITE (le code détermine la spécialité)

CODE_FOUR est l'identifiant de l'entité FOURNISSEUR. Il détermine tous les autres attributs de l'entité.

MDF pour l'association COMPOSE (RECETTE × INGREDIENT)

Pour l'association COMPOSE entre RECETTE et INGREDIENT, la clé est composée de (CODE_REC, CODE_ING) et détermine la quantité nécessaire :

(CODE_REC, CODE_ING) → QUANTITE

Cette dépendance fonctionnelle signifie que pour une recette donnée et un ingrédient donné, la quantité nécessaire est déterminée de manière unique.

Source \ Cible	CODE_REC	CODE_ING	QUANTITE	Total
CODE_REC	—	0	0	0
CODE_ING	0	—	0	0
(CODE_REC, CODE_ING)	0	0	1	**1**

MDF pour l'association CONSTITUTION (MENU × RECETTE)

Pour l'association CONSTITUTION entre les entités MENU et RECETTE, la clé est composée de (CODE_MENU, CODE_REC) et détermine l'ordre de service :

(CODE_MENU, CODE_REC) → ORDRE

Cette dépendance fonctionnelle signifie que pour un menu donné et une recette donnée, l'ordre de service est déterminé de manière unique.

(CODE_MENU, CODE_REC) est l'identifiant de l'association ‘’CONSTITUE’’. Il détermine l'attribut propre de l'association.

Source \ Cible	CODE_MENU	CODE_REC	ORDRE	Total
CODE_MENU	—	0	0	0
CODE_REC	0	—	0	0
(CODE_MENU, CODE_REC)	0	0	1	**1**

MDF pour l'association APPARTIENT (INGREDIENT × FOURNISSEUR)

Pour l'association APPARTIENT entre les entités INGREDIENT et FOURNISSEUR, la clé est CODE_ING et détermine le fournisseur :

CODE_ING → CODE_FOUR

Cette dépendance fonctionnelle signifie qu'un ingrédient appartient à un seul fournisseur principal. C'est une association de type un-à-plusieurs (1,n).

CODE_ING est l'identifiant de l'association APPARTIENT. Il détermine la clé étrangère vers l'entité FOURNISSEUR.

Source \ Cible	CODE_ING	CODE_FOUR	Total
CODE_ING	—	1	**1**
CODE_FOUR	0	—	0

Association REALISATION :

L'association REALISE relie les entités CHEF et RECETTE. Elle permet d'indiquer quels chefs sont capables de réaliser quelles recettes. Cette association est de type N,N, car un chef peut réaliser plusieurs recettes et une recette peut être réalisée par plusieurs chefs.

La dépendance fonctionnelle associée est :

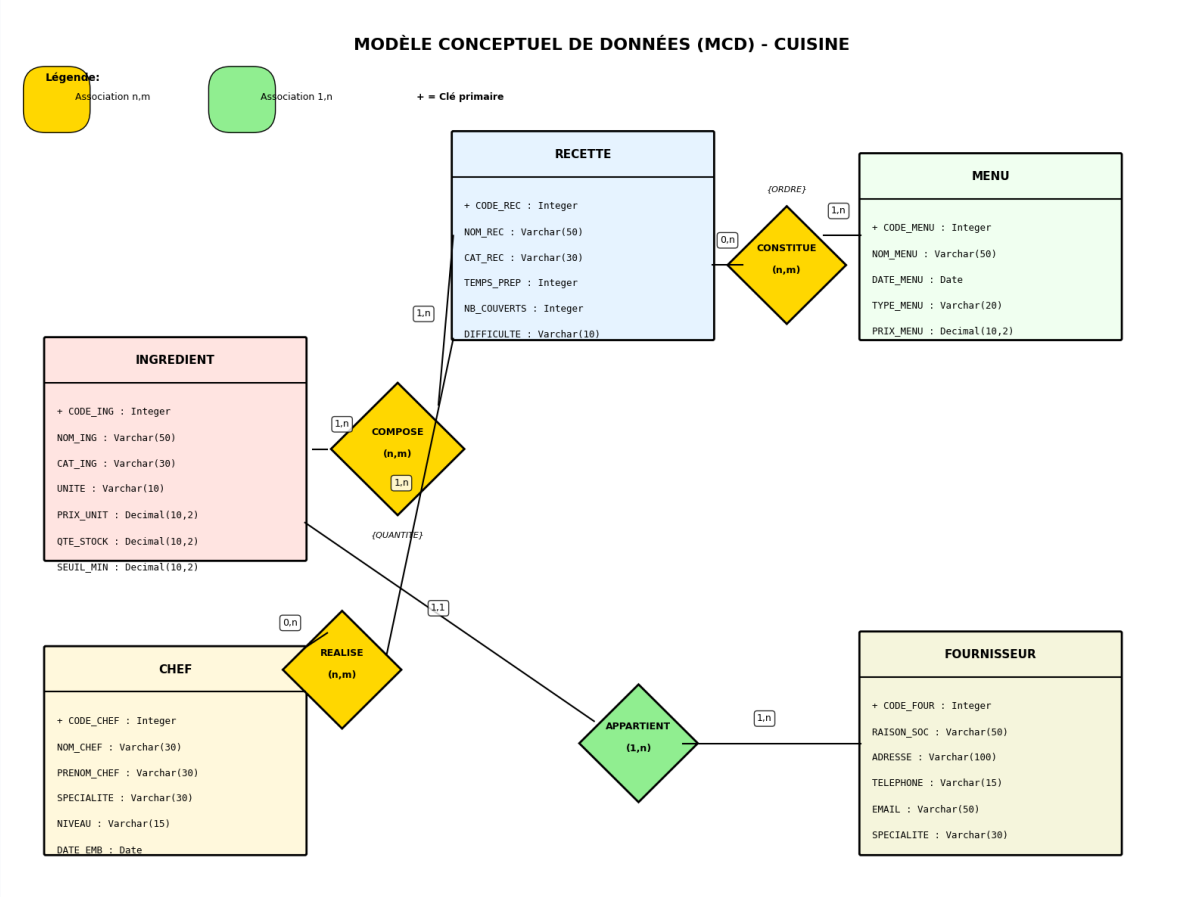
(CODE_CHEF, CODE_REC) → ∅

Cela signifie que le couple (CODE_CHEF, CODE_REC) ne détermine aucun attribut supplémentaire. En effet, cette association ne possède pas de propriétés propres telles qu'une date de réalisation, un niveau de maîtrise ou une durée d'exécution. Elle sert uniquement à représenter l'existence d'un lien entre un chef et une recette.

Ainsi, l'ensemble des attributs dépendants est vide, ce qui justifie la présence du symbole ∅. Cependant, pour implémenter cette relation dans le modèle logique, une table REALISATION est créée avec comme clé primaire le couple (CODE_CHEF, CODE_REC), permettant de matérialiser cette relation plusieurs-à-plusieurs.

Synthèse des Dépendances Fonctionnelles

Entité / Association	Dépendances Fonctionnelles
INGREDIENT	CODE_ING → NOM_ING, CAT_ING, UNITE, PRIX_UNIT, QTE_STOCK, SEUIL_MIN
RECETTE	CODE_REC → NOM_REC, CAT_REC, TEMPS_PREP, NB_COUVERTS, DIFFICULTE
MENU	CODE_MENU → NOM_MENU, DATE_MENU, TYPE_MENU, PRIX_MENU
CHEF	CODE_CHEF → NOM_CHEF, PRENOM_CHEF, SPECIALITE, NIVEAU, DATE_EMB
FOURNISSEUR	CODE_FOUR → RAISON_SOC, ADRESSE, TELEPHONE, EMAIL, SPECIALITE
COMPOSE	(CODE_REC, CODE_ING) → QUANTITE
CONSTITUE	(CODE_MENU, CODE_REC) → ORDRE
REALISE	(CODE_CHEF, CODE_REC) → ∅
APPARTIENT	CODE_ING → CODE_FOUR



Le MCD étant défini, nous passons maintenant au MLD, qui permet de transformer cette représentation conceptuelle en une structure logique directement exploitable dans une base de donnée

5. MODELE LOGIQUE DE DONNEES

Le passage du MCD au MLD a suivi les règles recommandées suivantes :

- Règle 1 : Chaque entité devient une table. L'identifiant de l'entité devient la clé primaire de la table.
- Règle 2 : Une association de type 1,n se traduit par l'ajout d'une clé étrangère dans la table du côté 1.
- Règle 3 : Une association de type n,m se traduit par la création d'une table intermédiaire. La clé primaire de cette table est composée des clés étrangères référençant les entités associées.

Application des règles :

Table INGREDIENT (CODE_ING, NOM_ING, CAT_ING, UNITE, PRIX_UNIT, QTE_STOCK, SEUIL_MIN, CODE_FOUR)

Clé primaire : CODE_ING

Clé étrangère : CODE_FOUR référence FOURNISSEUR

Table RECETTE (CODE_REC, NOM_REC, CAT_REC, TEMPS_PREP, NB_COUVERTS, DIFFICULTE)

Clé primaire : CODE_REC

Table MENU (CODE_MENU, NOM_MENU, DATE_MENU, TYPE_MENU, PRIX_MENU)

Clé primaire : CODE_MENU

Table CHEF (CODE_CHEF, NOM_CHEF, PRENOM_CHEF, SPECIALITE, NIVEAU, DATE_EMB)

Clé primaire : CODE_CHEF

Table FOURNISSEUR (CODE_FOUR, RAISON_SOC, ADRESSE, TELEPHONE, EMAIL, SPECIALITE)
Clé primaire : CODE_FOUR

Table COMPOSE (CODE_REC, CODE_ING, QUANTITE)
Clé primaire : (CODE_REC, CODE_ING)
Clés étrangères : CODE_REC référence RECETTE, CODE_ING référence INGREDIENT

Table CONSTITUTION (CODE_MENU, CODE_REC, ORDRE)
Clé primaire : (CODE_MENU, CODE_REC)
Clés étrangères : CODE_MENU référence MENU, CODE_REC référence RECETTE

Table REALISE (CODE_CHEF, CODE_REC)
Clé primaire : (CODE_CHEF, CODE_REC)

Le modèle logique étant établi, nous passons maintenant à la normalisation, qui permet d'améliorer la structure des tables et de garantir l'intégrité des données

6. NORMALISATION DE LA BASE

Lorsqu'une relation n'est pas correctement normalisée, plusieurs problèmes peuvent survenir :

- La redondance des données : Les mêmes informations sont stockées plusieurs fois, ce qui entraîne un gaspillage d'espace de stockage et des risques d'incohérence.
- Les anomalies de mise à jour :
- Anomalies d'insertion : Impossibilité d'insérer certaines données sans en insérer d'autres
- Anomalies de suppression : Perte d'informations lors de la suppression d'un tuple
- Anomalies de modification : Nécessité de modifier plusieurs tuples pour une seule information

En normalisant la base donnée de notre cuisine nous avons pour objectif de :

a. Éliminer les redondances inutiles de données

b. Éviter les anomalies de mise à jour

c. Assurer l'intégrité des données

d. Faciliter la maintenance de la base de données

e. Optimiser l'espace de stockage

La normalisation repose sur le concept de dépendance fonctionnelle et sur la décomposition des relations en relations plus petites et mieux structurées.
Nous allons suivre les étapes suivantes :

1. La relation universelle

La relation universelle est une relation qui regroupe tous les attributs du système d'information. Elle représente le point de départ du processus de normalisation.

2. Première Forme Normale (1FN)

Une relation est en Première Forme Normale si et seulement si tous ses attributs contiennent des valeurs atomiques (indivisibles).Cela signifie qu'à l'intersection d'une ligne et d'une colonne, on ne trouve qu'une seule valeur. Les attributs multivalués ou composés doivent être décomposés.

Exemple de non-1FN : Un attribut INGREDIENTS contenant « Farine, Sucre, Oeufs »
Solution : Créer une table séparée pour les ingrédients avec une ligne par ingrédient.

Toutes nos relations sont en 1FN car chaque attribut contient une valeur atomique.

3. Deuxième Forme Normale (2FN)

Une relation est en Deuxième Forme Normale si et seulement si :

- Elle est en 1FN
- Tout attribut non-clé dépend fonctionnellement de la totalité de la clé primaire (et pas seulement d'une partie)

La 2FN ne concerne que les relations ayant une clé composée. Une relation avec une clé simple est automatiquement en 2FN.

Exemple de non-2FN : Dans une table COMMANDE (NUM_CMD, CODE_ING, NOM_ING, QTE_CMD), NOM_ING ne dépend que de CODE_ING, pas de la totalité de la clé (NUM_CMD, CODE_ING).

Solution : Décomposer en deux tables : COMMANDE (NUM_CMD, CODE_ING, QTE_CMD) et INGREDIENT (CODE_ING, NOM_ING).

Dans notre MLD, la table COMPOSITION (CODE_REC, CODE_ING, QUANTITE) est en 2FN car QUANTITE dépend de la totalité de la clé composée.

4. Troisième Forme Normale (3FN)

Une relation est en Troisième Forme Normale si et seulement si :

- Elle est en 2FN
- Aucun attribut non-clé ne dépend fonctionnellement d'un autre attribut non-clé (pas de dépendance transitive)

La 3FN élimine les dépendances transitives qui créent des redondances.

Exemple de non-3FN : Dans une table EMPLOYE (CODE_EMP, NOM_EMP, CODE_DEPT, NOM_DEPT), NOM_DEPT dépend de CODE_DEPT qui dépend de CODE_EMP.

Solution : Décomposer en EMPLOYE (CODE_EMP, NOM_EMP, CODE_DEPT) et DEPARTEMENT (CODE_DEPT, NOM_DEPT).

Toutes nos relations sont en 3FN car il n'existe pas de dépendances transitives entre attributs non-clés.

Dans le cadre de cette étude, la normalisation a été effectuée jusqu'à la troisième forme normale (3FN). À ce niveau, toutes les dépendances fonctionnelles sont correctement prises en compte : chaque attribut non clé dépend entièrement de la clé primaire et uniquement de celle-ci. Cette étape permet déjà d'éliminer les

redondances principales et de prévenir les anomalies de mise à jour, d'insertion et de suppression.

5. FORME NORMALE DE BOYCE-CODD (BCNF).

Une relation est en BCNF si, pour toute dépendance fonctionnelle X → Y, X est une super-clé de la relation. Autrement dit, chaque déterminant doit être une clé candidate.

Dans cette étude, la base de données a été normalisée jusqu'en 3FN. Cela signifie que chaque information dépend entièrement de la clé primaire, ce qui réduit les doublons et évite les erreurs lors des ajouts, modifications ou suppressions.

On pourrait aller plus loin avec la BCNF, une forme plus stricte où chaque dépendance doit venir d'une clé. Toutefois, comme la structure en 3FN est déjà correcte, la BCNF n'apporte pas de correction supplémentaire importante ici.

6.1 PROCESSUS DE NORMALISATION APPLIQUE

1. Relation universelle

   Dans notre cas d'étude, une relation universelle pourrait être :

   CUISINE (CODE_ING, NOM_ING, CAT_ING, UNITE, PRIX_UNIT, QTE_STOCK, SEUIL_MIN, CODE_REC, NOM_REC, CAT_REC, TEMPS_PREP, NB_COUVERTS, DIFFICULTE, QUANTITE, CODE_MENU, NOM_MENU, DATE_MENU, TYPE_MENU, PRIX_MENU, CODE_CHEF, NOM_CHEF, PRENOM_CHEF, SPECIALITE, NIVEAU, DATE_EMB, CODE_FOUR, RAISON_SOC, ADRESSE, TELEPHONE, EMAIL)

Cette relation présente de nombreuses redondances et anomalies. Le processus de normalisation va la décomposer en relations plus petites et mieux structurées.

2. Vérification de la 1FN

Vérifions que toutes nos relations sont en 1FN :

- INGREDIENT : Tous les attributs sont atomiques
- RECETTE : Tous les attributs sont atomiques
- MENU : Tous les attributs sont atomiques
- CHEF : Tous les attributs sont atomiques
- FOURNISSEUR : Tous les attributs sont atomiques
- COMPOSITION : Tous les attributs sont atomiques
- CONSTITUTION : Tous les attributs sont atomiques
- REALISATION : Tous les attributs sont atomiques

Conclusion : Toutes les relations sont en 1FN.

3. Vérification de la 2FN

Vérifions que toutes nos relations sont en 2FN :

Relations avec clé simple (automatiquement en 2FN) : INGREDIENT, RECETTE, MENU, CHEF, FOURNISSEUR.

Relations avec clé composée :

- COMPOSITION (CODE_REC, CODE_ING, QUANTITE) : QUANTITE dépend de la totalité de la clé → 2FN respectée

- CONSTITUTION (CODE_MENU, CODE_REC, ORDRE) : ORDRE dépend de la totalité de la clé → 2FN respectée

- REALISATION (CODE_CHEF, CODE_REC) : Pas d'attribut non-clé → 2FN respectée

Conclusion : Toutes les relations sont en 2FN.

4. Vérification de la 3FN

Vérifions que toutes nos relations sont en 3FN :

INGREDIENT : CODE_ING → tous les attributs. Pas de dépendance entre attributs non-clés → 3FN respectée.

RECETTE : CODE_REC → tous les attributs. Pas de dépendance entre attributs non-clés → 3FN respectée.

MENU : CODE_MENU → tous les attributs. Pas de dépendance entre attributs non-clés → 3FN respectée.

CHEF : CODE_CHEF → tous les attributs. Pas de dépendance entre attributs non-clés → 3FN respectée.

FOURNISSEUR : CODE_FOUR → tous les attributs. Pas de dépendance entre attributs non-clés → 3FN respectée.

Conclusion : Toutes les relations sont en 3FN.

Schéma relationnel normalisé

Notre schéma relationnel final, entièrement normalisé, est le suivant :

INGREDIENT (CODE_ING, NOM_ING, CAT_ING, UNITE, PRIX_UNIT, QTE_STOCK, SEUIL_MIN, CODE_FOUR)

Clé primaire : CODE_ING

Clé étrangère : CODE_FOUR référence FOURNISSEUR

RECETTE (CODE_REC, NOM_REC, CAT_REC, TEMPS_PREP, NB_COUVERTS, DIFFICULTE)
Clé primaire : CODE_REC

MENU (CODE_MENU, NOM_MENU, DATE_MENU, TYPE_MENU, PRIX_MENU)
Clé primaire : CODE_MENU

CHEF (CODE_CHEF, NOM_CHEF, PRENOM_CHEF, SPECIALITE, NIVEAU, DATE_EMB)
Clé primaire : CODE_CHEF

FOURNISSEUR (CODE_FOUR, RAISON_SOC, ADRESSE, TELEPHONE, EMAIL, SPECIALITE)
Clé primaire : CODE_FOUR

COMPOSE (CODE_REC, CODE_ING, QUANTITE)
Clé primaire : (CODE_REC, CODE_ING)
Clés étrangères : CODE_REC référence RECETTE, CODE_ING référence INGREDIENT

CONSTITUE (CODE_MENU, CODE_REC, ORDRE)
Clé primaire : (CODE_MENU, CODE_REC)
Clés étrangères : CODE_MENU référence MENU, CODE_REC référence RECETTE

REALISE (CODE_CHEF, CODE_REC)

Clé primaire : (CODE_CHEF, CODE_REC)

Clés étrangères : CODE_CHEF référence CHEF, CODE_REC référence RECETTE

Après avoir terminé la phase de conception et de normalisation de la base de données jusqu'à la 3FN, nous pouvons maintenant passer à l'étape d'implémentation. Celle-ci consiste à traduire le modèle obtenu en scripts SQL afin de créer les tables, définir les clés primaires et étrangères, et mettre en place les contraintes d'intégrité. Cette phase permet de rendre la base de données opérationnelle et exploitable dans un système de gestion de base de données

7. ÉVALUATION DES SYSTÈMES DE GESTION DE BASES DE DONNÉES POUR LE PROJET CUISINE

Le choix d'un Système de Gestion de Base de Données constitue une décision stratégique pour la réussite du projet de gestion de cuisine professionnelle. Cette analyse compare les principales solutions disponibles selon des critères pertinents pour notre contexte. Les critères d'évaluation retenus sont les suivants : le coût de licence et de déploiement, la facilité d'administration et de maintenance, les performances pour notre volumétrie prévue, la conformité aux standards SQL, la robustesse des transactions ACID, l'évolutivité future du projet, la disponibilité de l'hébergement et enfin la richesse des fonctionnalités analytiques.

- MySQL

MySQL représente le SGBD relationnel open source le plus répandu mondialement. Il est actuellement maintenu par Oracle Corporation qui propose une version Community gratuite sous licence GPL ainsi qu'une version Enterprise payante.

Les avantages de MySQL sont nombreux pour notre projet. Premièrement, le coût est nul pour la version Community ce qui correspond parfaitement au budget d'une PME de restauration. Deuxièmement, la documentation est extrêmement abondante avec des tutoriels accessibles pour toutes les compétences. Troisièmement, l'intégration avec les langages de programmation est universelle que ce soit PHP, Python, Java ou Node.js. Quatrièmement, l'hébergement web mutualisé propose quasi systématiquement MySQL ce qui facilite le déploiement. Cinquièmement, la courbe d'apprentissage est relativement douce pour une équipe technique modeste.

Les inconvénients méritent attention. La direction commerciale d'Oracle crée une incertitude sur l'évolution future du produit. Les fonctionnalités avancées comme les expressions de table communes récursives ou les fonctions de fenêtrage ont été historiquement limitées comparées à PostgreSQL bien que la version 8.0 ait comblé beaucoup d'écarts. Le moteur MyISAM historique ne supporte pas les transactions ACID ce qui impose une configuration stricte en InnoDB. Enfin, MySQL est historiquement moins strict sur les contraintes d'intégrité référentielle ce qui demande vigilance lors du développement.

Pour notre projet cuisine, MySQL présente un excellent rapport coût-benéfice avec une facilité de mise en œuvre remarquable. Il convient parfaitement aux requêtes standards de gestion des stocks et des recettes. La vigilance porte sur la configuration InnoDB obligatoire pour garantir l'intégrité des transactions de stock.

- PostgreSQL

PostgreSQL se positionne comme le SGBD relationnel open source le plus avancé techniquement avec une réputation d'excellence en matière de robustesse et de conformité aux standards.

Les forces de PostgreSQL sont déterminantes pour notre usage. La conformité aux standards SQL est la meilleure du marché ce qui assure la portabilité des compétences et des requêtes. Les fonctionnalités sont exceptionnellement riches incluant les expressions de table communes récursives, les fonctions de fenêtrage, le support natif du format JSON et les extensions géospatiales. La robustesse des transactions ACID est reconnue industrielle avec une fiabilité éprouvée dans les environnements critiques. L'extensibilité permet de créer des types personnalisés, des fonctions et d'ajouter des extensions selon les besoins futurs. La licence PostgreSQL est très permissive sans risque de capture propriétaire.

Les limitations concernent principalement la complexité relative. La courbe d'apprentissage est plus steep que MySQL notamment pour l'administration et

l'optimisation. La consommation mémoire est supérieure ce qui peut impacter les petits serveurs. L'hébergement mutualisé classique propose moins souvent PostgreSQL que MySQL bien que cette tendance s'inverse. L'administration fine demande une expertise plus poussée pour le tuning des performances.

Pour notre projet cuisine, PostgreSQL excelle particulièrement sur les aspects analytiques. Le calcul des coûts de revient par recette, les statistiques de consommation, les tendances d'approvisionnement bénéficient des fonctions analytiques avancées. La gestion des stocks critique trouve dans PostgreSQL un garant de fiabilité transactionnelle. L'absence de risque de licence propriétaire sécurise l'investissement long terme.

- MariaDB

MariaDB constitue un fork de MySQL créé par les fondateurs originaux de MySQL après le rachat par Oracle. Elle se positionne comme une alternative purement open source garantie.

Les atouts de MariaDB résident dans sa compatibilité quasi totale avec MySQL permettant une migration transparente dans les deux sens. L'engagement open source est total sans version Enterprise payante contrairement à MySQL. Quelques extensions par rapport à MySQL sont disponibles comme le moteur de colonnes ColumnStore pour l'analytique. Les performances sont globalement comparables à MySQL avec parfois des avantages sur certaines charges.

Les points d'attention incluent une communauté légèrement plus restreinte que MySQL ce qui se traduit par une documentation moins abondante. L'écosystème d'outils et d'hébergeurs spécialisés est moins développé. La divergence progressive avec MySQL introduit des fonctionnalités non compatibles ce qui peut compliquer les migrations futures.

Pour notre projet, MariaDB représente une alternative viable à MySQL si l'on souhaite éviter toute dépendance à Oracle. Le choix entre MySQL et MariaDB dépendra davantage de considérations philosophiques sur l'open source que de différences techniques majeures pour notre usage.

- SQLite

SQLite se distingue comme un SGBD embarqué sans serveur stocké dans un simple fichier local.

Les avantages sont la simplicité extrême avec aucune installation serveur requise. La portabilité est maximale avec un fichier unique multiplateforme. L'empreinte ressource est minimale de l'ordre de quelques centaines de kilo-octets. L'intégration dans les applications est native notamment en Python et sur mobile. Le domaine public libre de tout droit est un atout juridique.

Les limitations sont bloquantes pour notre contexte. La capacité multi-utilisateurs est faible avec des verrous gros-bloc sur les écritures. Les volumes au-delà de quelques gigaoctets deviennent problématiques. Les fonctionnalités avancées sont absentes : pas de stored procedures, triggers limités, types peu stricts. L'accès réseau client-serveur natif n'existe pas. Les outils d'administration sont rudimentaires.

Pour notre projet cuisine multi-utilisateurs avec cuisine et bureau simultanés, SQLite est inadapté. Il pourrait convenir uniquement pour un déploiement strictement monoposte ou une application mobile autonome sans synchronisation temps réel.

- Microsoft SQL Server Express

SQL Server Express représente la version gratuite mais limitée du SGBD propriétaire leader en environnement enterprise Microsoft.

Les qualités incluent des outils d'administration exceptionnels avec SQL Server Management Studio reconnu comme référence. L'intégration avec l'écosystème Microsoft est parfaite pour les environnements .NET. L'optimiseur de requêtes est très avancé. Les fonctionnalités de sécurité sont robustes.

Les contraintes sont significatives. La limitation à 10 gigaoctets de données est restrictive pour une croissance future. La plateforme est historiquement Windows bien qu'une version Linux existe désormais mais moins mature. La consommation ressource est élevée en mémoire et processeur. La version complète est très coûteuse en cas de dépassement des limites Express. L'hébergement cloud est relativement cher comparé aux solutions open source.

Pour notre projet de cuisine professionnelle, SQL Server Express apparaît surdimensionné et verrouillé dans un écosystème propriétaire. Il ne se justifie que si l'infrastructure Microsoft existe déjà en place.

- Oracle Database Express Edition

Oracle XE constitue la version gratuite mais très limitée du SGBD leader du marché enterprise.

Les forces incluent une robustesse légendaire et un optimiseur de requêtes exceptionnel. Les fonctionnalités sont très complètes même dans la version Express.

Les faiblesses sont nombreuses pour notre contexte. La complexité d'installation et d'administration est extrême. La limitation à 2 processeurs, 2 gigaoctets de RAM et 12 gigaoctets de données est rapidement contraignante. La consommation minimale de 2 gigaoctets RAM est élevée. La montée de version vers l'édition complète est financièrement prohibitive pour une PME. L'expertise requise est spécialisée et coûteuse.

Pour notre projet cuisine, Oracle XE est clairement exclu comme tout produit Oracle par disproportion entre la complexité et les besoins réels.

- MongoDB

MongoDB est le représentant le plus connu des SGBD NoSQL orientés documents avec stockage JSON-like.

Les avantages sont la flexibilité de schéma sans migration structurale. Le stockage JSON natif correspond aux formats modernes d'échange. La réplication et le sharding sont natifs pour la haute disponibilité. Le scaling horizontal est facilité.

Les inconvénients sont majeurs pour notre modèle. L'absence de jointures SQL natives complique les requêtes entre recettes et ingrédients. Les transactions multi-documents sont récentes et moins matures qu'en relationnel. Le modèle de cohérence éventuelle par défaut ne garantit pas la stricte intégrité des stocks. Les agrégations complexes sont moins intuitives que SQL pour les équipes formées au relationnel.

Pour notre projet cuisine intrinsèquement relationnel avec de nombreuses jointures nécessaires, MongoDB est inadapté. Il ne se justifierait que dans une architecture microservices préexistante avec des services découplés.

SYNTHÈSE COMPARATIVE

Classement par critère

Critère Coût Premier ex aequo : MySQL, PostgreSQL, MariaDB, SQLite tous gratuits Dernier : Oracle puis SQL Server en versions complètes très coûteuses

Critère Facilité Premier : SQLite par sa simplicité extrême Deuxième : MySQL et MariaDB par leur courbe d'apprentissage douce Troisième : PostgreSQL par sa richesse fonctionnelle compensant sa complexité Derniers : SQL Server et Oracle par leur lourdeur administrative

Critère Robustesse transactionnelle Premier ex aequo : PostgreSQL et Oracle Deuxième : SQL Server Troisième : MySQL et MariaDB avec InnoDB configuré correctement Dernier : SQLite par ses limitations de concurrence

Critère Fonctionnalités analytiques Premier : PostgreSQL par ses fonctions de fenêtrage et CTE avancées Deuxième : Oracle et SQL Server Troisième : MySQL version 8.0+ ayant comblé son retard Dernier : SQLite très limité

Critère Évolutivité Premier : PostgreSQL par sa capacité à monter en charge sans changement technologique Deuxième : MySQL et MariaDB correctement configurés Troisième : MongoDB pour le scaling horizontal bien que mal adapté à notre modèle Dernier : SQLite bloqué par sa nature embarquée

RECOMMANDATION FINALE

Choix optimal : PostgreSQL

Après analyse complète, PostgreSQL émerge comme le choix optimal pour le projet de base de données cuisine. Les arguments fondant cette recommandation sont structurés selon plusieurs dimensions.

Dimension technique La conformité stricte aux standards SQL assure la pérennité des développements et la portabilité des compétences. Les transactions ACID robustes garantissent l'intégrité des données critiques de stock et de coûts. Les fonctionnalités analytiques avancées permettent les calculs de coûts de revient complexes et les statistiques de gestion. L'optimiseur de requêtes de qualité industrielle assure de bonnes performances sans tuning excessif.

Dimension économique L'absence totale de coût de licence élimine tout risque budgétaire pour la PME. L'absence de risque de capture propriétaire sécurise l'investissement long terme contrairement à MySQL sous Oracle. Les compétences PostgreSQL sont transférables et valorisables sur le marché du travail.

Dimension opérationnelle La réputation de fiabilité éprouvée en production réduit les risques d'indisponibilité critique. L'extensibilité par extensions permet d'enrichir fonctionnellement sans changer de SGBD. La communauté active assure un support et une veille technique continus.

Alternative acceptable : MySQL ou MariaDB

Si l'équipe projet possède déjà une expertise MySQL significative, cette solution reste tout à fait viable. Les précautions à prendre sont les suivantes : configurer impérativement le moteur InnoDB pour les transactions ACID, activer le mode SQL strict pour garantir l'intégrité des données, privilégier la version 8.0 ou supérieure pour bénéficier des fonctionnalités analytiques améliorées. MariaDB constitue une alternative préférable à MySQL si l'on souhaite éviter toute dépendance à Oracle tout en conservant une compatibilité technique.

Solutions à écarter

SQLite est à écarter pour tout déploiement multi-utilisateurs impliquant simultanément la cuisine et le bureau de gestion. Ses limitations de concurrence en écriture créeraient des conflits et des risques de corruption.

Oracle Database est à écarter par disproportion flagrante entre la complexité administrative et les besoins modestes du projet. Le coût et la expertise requise sont injustifiés.

MongoDB est à écarter car le modèle de données cuisine est intrinsèquement relationnel avec de nombreuses associations many-to-many entre recettes et ingrédients, menus et recettes. La modélisation document serait contournée et inefficace.

ARCHITECTURE RECOMMANDÉE

L'architecture cible avec PostgreSQL s'organise en trois couches distinctes.

La couche présentation comprend l'application web pour le bureau de gestion développée en React ou Vue.js et l'application mobile pour la cuisine en React Native ou Flutter.

La couche applicative expose une API backend en Node.js, Python ou Java utilisant un ORM moderne comme Sequelize, TypeORM, Prisma ou SQLAlchemy pour l'abstraction de la base de données.

La couche données est assurée par PostgreSQL 14 ou supérieur avec les extensions potentielles suivantes : pg_trgm pour la recherche textuelle floue dans les noms de recettes, PostGIS si géolocalisation des fournisseurs utile, TimescaleDB si historique temporel détaillé des variations de stock souhaité.

La sauvegarde et le monitoring s'appuient sur pgBackRest pour les sauvegardes physiques, pgAdmin pour l'administration graphique, et Prometheus avec Grafana pour le monitoring des performances.

Le choix d'un SGBD pour le projet cuisine doit équilibrer robustesse technique, coût maîtrisé et adéquation au modèle relationnel. PostgreSQL offre le meilleur compromis global avec une excellence technique sans coût de licence. MySQL ou MariaDB constituent des alternatives viables si les compétences internes le justifient. Les solutions SQLite, Oracle et MongoDB sont inadaptées respectivement par insuffisance de concurrence, excès de complexité et inadéquation au modèle relationnel.

L'investissement dans PostgreSQL forme une fondation solide pour l'évolution future du projet vers de la business intelligence, du mobile synchronisé ou du multi-établissement.

Les ressources du projet sont disponibles sur le dépôt GitHub :

https://github.com/devoirbdiai2025-arch/DEVOIR_BD.git

8. IMPLÉMENTATION SQL

CATÉGORIE 1 : REQUÊTES DE SÉLECTION

Extraction de données sans modification - fondamentaux du langage SQL.

8.1 R1.1 : Sélection complète

Code SQL :

```
SELECT * FROM INGREDIENT;
```

Application : Inventaire complet des produits

8.2 R1.2 : Colonnes spécifiques

Code SQL :

```
SELECT NOM_ING, PRIX_UNIT FROM INGREDIENT;
```

Application : Liste tarifaire simplifiée

8.3 R1.3 : Alias de colonnes

Code SQL :

```
SELECT NOM_ING AS Produit, PRIX_UNIT AS Prix FROM INGREDIENT;
```

Application : Affichage utilisateur francisé

8.4 R1.4 : Suppression des doublons

Code SQL :

```
SELECT DISTINCT CAT_ING FROM INGREDIENT;
```

Application : Liste des catégories disponibles

8.5 R1.5 : Tri des résultats

Code SQL :

```
SELECT * FROM INGREDIENT ORDER BY PRIX_UNIT DESC;
```

Application : Produits classés par prix décroissant

8.6 R1.6 : Limitation du nombre

Code SQL :

```
SELECT * FROM INGREDIENT LIMIT 10;
```

Application : Aperçu des 10 premiers produits

CATÉGORIE 2 : FILTRAGE DE DONNÉES

Utilisation de la clause WHERE pour filtrer les enregistrements.

8.7 R2.1 : Égalité

Code SQL :

```
SELECT * FROM INGREDIENT WHERE CAT_ING = 'Viandes';
```

Application : Sélection des viandes uniquement

8.8 R2.2 : Différence

Code SQL :

```
SELECT * FROM INGREDIENT WHERE CAT_ING != 'Légumes';
```

Application : Exclusion des légumes

8.9 R2.3 : Comparaison numérique

Code SQL :

```
SELECT * FROM INGREDIENT WHERE PRIX_UNIT > 1000;
```

Application : Produits de plus de 1000 FCFA

8.10 R2.4 : Intervalle

Code SQL :

```
SELECT * FROM INGREDIENT WHERE PRIX_UNIT BETWEEN 500 AND 1500;
```

Application : Fourchette de prix spécifique

8.11 R2.5 : Liste de valeurs

Code SQL :

```
SELECT * FROM INGREDIENT WHERE CAT_ING IN ('Viandes', 'Poissons');
```

Application : Sélection multiple de catégories

8.12 R2.6 : Recherche par motif

Code SQL :

```
SELECT * FROM INGREDIENT WHERE NOM_ING LIKE '%sauce%';
```

Application : Recherche textuelle dans les noms

8.13 R2.7 : Test de nullité

Code SQL :

```
SELECT * FROM CHEF WHERE DATE_EMB IS NULL;
```

Application : Chefs sans date d'embauche enregistrée

8.14 R2.8 : Conditions combinées (ET)

Code SQL :

```
SELECT * FROM INGREDIENT WHERE CAT_ING = 'Légumes' AND PRIX_UNIT < 1000;
```

Application : Légumes bon marché

8.15 R2.9 : Conditions combinées (OU)

Code SQL :

```
SELECT * FROM INGREDIENT WHERE CAT_ING = 'Viandes' OR CAT_ING = 'Poissons';
```

Application : Toutes les protéines animales

CATÉGORIE 3 : JOINTURES ENTRE TABLES

Association de données provenant de plusieurs tables relationnelles.

8.16 R3.1 : Jointure interne

Code SQL :

```
SELECT I.NOM_ING, F.RAISON_SOC
FROM INGREDIENT I
JOIN FOURNISSEUR F ON I.CODE_FOUR = F.CODE_FOUR;
```

Application : Ingrédients avec leurs fournisseurs

8.17 R3.2 : Jointure avec alias

Code SQL :

```
SELECT
   I.NOM_ING                                    AS                                    Produit,
   I.PRIX_UNIT                                    AS                                    Prix,
   F.RAISON_SOC                                 AS                                 Fournisseur
FROM                                        INGREDIENT                                        I
JOIN FOURNISSEUR F ON I.CODE_FOUR = F.CODE_FOUR;
```

Application : Affichage professionnel des approvisionnements

8.18 R3.3 : Jointure multiple (3 tables)

Code SQL :

```
SELECT
   R.NOM_REC                                    AS                                    Recette,
   I.NOM_ING                                   AS                                  Ingredient,
   C.QUANTITE
FROM                                         RECETTE                                         R
JOIN          COMPOSITION          C          ON          R.CODE_REC          =          C.CODE_REC
JOIN INGREDIENT I ON C.CODE_ING = I.CODE_ING;
```

Application : Composition détaillée des recettes

8.19 R3.4 : Jointure externe gauche

Code SQL :

```
SELECT          F.RAISON_SOC,          COUNT(I.CODE_ING)          AS          NbIngredients
FROM                                   FOURNISSEUR                                        F
LEFT        JOIN        INGREDIENT        I        ON        F.CODE_FOUR        =        I.CODE_FOUR
GROUP BY F.CODE_FOUR, F.RAISON_SOC;
```

Application : Fournisseurs même sans produits associés

CATÉGORIE 4 : AGRÉGATION ET STATISTIQUES

Calculs groupés et analyses statistiques avec GROUP BY.

8.20 R4.1 : Comptage

Code SQL :

```
SELECT CAT_ING, COUNT(*) AS NbIngredients FROM INGREDIENT GROUP BY CAT_ING;
```

Application : Nombre de produits par catégorie

8.21 R4.2 : Somme

Code SQL :

```
SELECT SUM(QTE_STOCK * PRIX_UNIT) AS ValeurTotale FROM INGREDIENT;
```

Application : Valorisation totale du stock en FCFA

8.22 R4.3 : Moyenne

Code SQL :

```
SELECT CAT_ING, AVG(PRIX_UNIT) AS PrixMoyen FROM INGREDIENT GROUP BY CAT_ING;
```

Application : Prix moyen par famille de produits

8.23 R4.4 : Min/Max

Code SQL :

```
SELECT MIN(PRIX_UNIT) AS PrixMin, MAX(PRIX_UNIT) AS PrixMax FROM INGREDIENT;
```

Application : Fourchette de prix du stock

8.24 R4.5 : Filtrage de groupes

Code SQL :

```
SELECT CAT_ING, COUNT(*) AS Nb FROM INGREDIENT GROUP BY CAT_ING HAVING COUNT(*) > 5;
```

Application : Catégories avec large assortiment (>5)

8.25 R4.6 : Statistiques complètes

Code SQL :

```
SELECT
   CAT_ING,
   COUNT(*)                    AS                    Nb,
   SUM(QTE_STOCK)              AS              QteTotale,
   AVG(PRIX_UNIT)              AS               PrixMoy,
   MIN(PRIX_UNIT)              AS               PrixMin,
   MAX(PRIX_UNIT)              AS               PrixMax
FROM                                           INGREDIENT
GROUP BY CAT_ING;
```

Application : Tableau de bord complet par catégorie

CATÉGORIE 5 : MODIFICATION DE DONNÉES

Mises à jour avec sécurité - toujours vérifier avec SELECT avant.

⚠ ATTENTION : Toujours utiliser WHERE pour éviter les modifications massives accidentelles.

8.26 R5.1 : Mise à jour simple

Code SQL :

```
UPDATE INGREDIENT SET PRIX_UNIT = 1300 WHERE NOM_ING = 'Farine de blé';
```

Application : Correction de prix unitaire

8.27 R5.2 : Mise à jour calculée

Code SQL :

```
UPDATE INGREDIENT SET PRIX_UNIT = PRIX_UNIT * 1.10 WHERE CAT_ING = 'Viandes';
```

Application : Augmentation de 10% sur les viandes

8.28 R5.3 : Mise à jour multi-colonnes

Code SQL :

```
UPDATE CHEF
SET NIVEAU = 'Sous-chef', SPECIALITE = 'Plat'
WHERE NOM_CHEF = 'Kamdem';
```

Application : Promotion et changement d'affectation

8.29 R5.4 : Mise à jour avec jointure

Code SQL :

```
UPDATE INGREDIENT I
SET PRIX_UNIT = PRIX_UNIT * 1.05
FROM FOURNISSEUR F
WHERE I.CODE_FOUR = F.CODE_FOUR AND F.SPECIALITE = 'Légumes';
```

Application : Répercussion hausse fournisseur légumes

CATÉGORIE 6 : SUPPRESSION DE DONNÉES

⚠ ATTENTION CRITIQUE : La suppression est irréversible. Préférer un statut "inactif" en production.

8.30 R6.1 : Suppression ciblée

Code SQL :

```
DELETE FROM INGREDIENT WHERE CODE_ING = 40;
```

Application : Retrait produit obsolète (Sang de bœuf)

8.31 R6.2 : Suppression conditionnelle

Code SQL :

```
DELETE FROM INGREDIENT WHERE QTE_STOCK = 0;
```

Application : Nettoyage des ruptures de stock

8.32 R6.3 : Suppression avec sous-requête

Code SQL :

```
DELETE FROM INGREDIENT
WHERE CODE_FOUR IN (
  SELECT CODE_FOUR FROM FOURNISSEUR WHERE SPECIALITE = 'Chocolat'
);
```

Application : Suppression par type de fournisseur

CATÉGORIE 7 : AJOUT DE DONNÉES

8.33 R7.1 : Insertion complète

Code SQL :

```
INSERT INTO FOURNISSEUR VALUES (9, 'Nouveau Fournisseur', 'Adresse', '+237 600 00 00 00', 'email@test.cm', 'Spécialité');
```

Application : Nouveau partenaire commercial

8.34 R7.2 : Insertion partielle

Code SQL :

```
INSERT INTO INGREDIENT (CODE_ING, NOM_ING, CAT_ING, UNITE, PRIX_UNIT) VALUES (41, 'Nouveau', 'Légumes', 'kg', 500);
```

Application : Ajout avec valeurs par défaut

8.35 R7.3 : Insertion multiple

Code SQL :

```
INSERT INTO INGREDIENT (CODE_ING, NOM_ING, CAT_ING) VALUES
(42, 'Ingrédient A', 'Fruits'),
(43, 'Ingrédient B', 'Viandes'),
(44, 'Ingrédient C', 'Épices');
```

Application : Batch d'ajouts simultanés

8.36 R7.4 : Insertion par copie

Code SQL :

```
INSERT INTO INGREDIENT (CODE_ING, NOM_ING, CAT_ING, UNITE, PRIX_UNIT)
SELECT CODE_ING + 100, NOM_ING || ' (Copie)', CAT_ING, UNITE, PRIX_UNIT
FROM INGREDIENT WHERE CAT_ING = 'Légumes';
```

Application : Duplication d'une catégorie

CATÉGORIE 8 : SOUS-REQUÊTES

Requêtes imbriquées pour traitements complexes.

8.37 R8.1 : Sous-requête scalaire

Code SQL :

```
SELECT NOM_ING,
  (SELECT AVG(PRIX_UNIT) FROM INGREDIENT) AS PrixMoyenGeneral
FROM INGREDIENT;
```

Application : Comparaison avec moyenne globale

8.38 R8.2 : Sous-requête corrélée

Code SQL :

```
SELECT NOM_ING, PRIX_UNIT
FROM INGREDIENT I
WHERE PRIX_UNIT > (
  SELECT AVG(PRIX_UNIT) FROM INGREDIENT WHERE CAT_ING = I.CAT_ING
);
```

Application : Produits chers dans leur catégorie

8.39 R8.3 : Opérateur IN

Code SQL :

```
SELECT * FROM RECETTE
WHERE CODE_REC IN (
  SELECT CODE_REC FROM COMPOSITION WHERE CODE_ING = 6
);
```

Application : Recettes avec banane plantain

8.40 R8.4 : Opérateur EXISTS

Code SQL :

```
SELECT * FROM RECETTE R
WHERE EXISTS (
  SELECT 1 FROM COMPOSITION C WHERE C.CODE_REC = R.CODE_REC
);
```

Application : Recettes ayant des ingrédients définis

8.41 R8.5 : Opérateur NOT EXISTS

Code SQL :

```
SELECT * FROM INGREDIENT I
WHERE NOT EXISTS (
  SELECT 1 FROM COMPOSITION C WHERE C.CODE_ING = I.CODE_ING
);
```

Application : Produits non utilisés en cuisine

CATÉGORIE 9 : OPÉRATIONS ENSEMBLISTES

8.42 R9.1 : Union sans doublon

Code SQL :

```
SELECT NOM_ING FROM INGREDIENT WHERE CAT_ING = 'Viandes'
UNION
SELECT NOM_ING FROM INGREDIENT WHERE CAT_ING = 'Poissons';
```

Application : Liste unique des protéines

8.43 R9.2 : Union avec doublons

Code SQL :

```
SELECT NOM_CHEF FROM CHEF WHERE SPECIALITE = 'Plat'
UNION ALL
SELECT NOM_CHEF FROM CHEF WHERE NIVEAU = 'Chef de partie';
```

Application : Chefs qualifiés (comptage multiple possible)

8.44 R9.3 : Intersection

Code SQL :

```
SELECT CODE_CHEF FROM CHEF WHERE SPECIALITE = 'Plat'
INTERSECT
SELECT CODE_CHEF FROM REALISATION WHERE CODE_REC = 5;
```

Application : Chefs plats sachant faire Poulet DG

8.45 R9.4 : Différence

Code SQL :

```
SELECT CODE_ING FROM INGREDIENT
EXCEPT
SELECT CODE_ING FROM COMPOSITION;
```

Application : Produits jamais utilisés en recette

CATÉGORIE 10 : FONCTIONS DE FENÊTRAGE

Analyses avancées avec fonctions analytiques (PostgreSQL).

8.46 R10.1 : Numérotation

Code SQL :

```
SELECT
  NOM_ING,
  PRIX_UNIT,
  ROW_NUMBER() OVER (ORDER BY PRIX_UNIT DESC) AS Rang
FROM INGREDIENT;
```

Application : Classement simple des produits

8.47 R10.2 : Classement avec ex-aequo

Code SQL :

```
SELECT
  NOM_ING,
  PRIX_UNIT,
```

```
    RANK() OVER (ORDER BY PRIX_UNIT DESC) AS Rang
FROM INGREDIENT;
```

Application : Gestion des égalités de prix

8.48 R10.3 : Classement par groupe

Code SQL :

```
SELECT
    CAT_ING,
    NOM_ING,
    PRIX_UNIT,
    DENSE_RANK() OVER (PARTITION BY CAT_ING ORDER BY PRIX_UNIT DESC) AS Rang
FROM INGREDIENT;
```

Application : Classement dans chaque catégorie

8.49 R10.4 : Navigation

Code SQL :

```
SELECT
    NOM_ING,
    PRIX_UNIT,
    LAG(PRIX_UNIT) OVER (ORDER BY PRIX_UNIT) AS Precedent,
    LEAD(PRIX_UNIT) OVER (ORDER BY PRIX_UNIT) AS Suivant
FROM INGREDIENT;
```

Application : Comparaison avec produits voisins

8.50 R10.5 : Somme cumulative

Code SQL :

```
SELECT
    NOM_ING,
    PRIX_UNIT,
    SUM(PRIX_UNIT) OVER (ORDER BY PRIX_UNIT) AS Cumul
FROM INGREDIENT;
```

Application : Calcul progressif des valeurs

CATÉGORIE 11 : VUES PRÉDÉFINIES

Tables virtuelles pour accès simplifié aux analyses courantes.

8.51 R11.1 : Vue stock critique

Code SQL :

```
SELECT * FROM V_STOCK_CRITIQUE ORDER BY MANQUE DESC;
```

Application : Alertes réapprovisionnement urgent

8.52 R11.2 : Vue carte recettes

Code SQL :

```
SELECT * FROM V_CARTE_RECETTES WHERE COUT_TOTAL_FCFA > 5000;
```

Application : Recettes coûtant plus de 5000 FCFA

8.53 R11.3 : Vue planning

Code SQL :

```
SELECT * FROM V_PLANNING_MENUS WHERE DATE_MENU = '2026-04-20';
```

Application : Menu du jour détaillé

8.54 R11.4 : Vue compétences

Code SQL :

```
SELECT * FROM V_COMPETENCES_CHEFS ORDER BY NB_RECETTES_MAITRISEES DESC;
```

Application : Classement chefs par polyvalence

CATÉGORIE 12 : FONCTIONS STOCKÉES

Traitements encapsulés pour utilisation simplifiée.

8.55 R12.1 : Calcul rentabilité menu

Code SQL :

```
SELECT * FROM calculer_cout_menu(1);
```

Application : Analyse économique Menu Tradition Bamiléké

Retour : Retourne : nom_menu, prix_vente, cout_total, marge

8.56 R12.2 : Génération liste courses

Code SQL :

```
SELECT * FROM liste_courses();
```

Application : Liste automatique des achats nécessaires

Retour : Retourne : ingredient, categorie, quantite, unite, fournisseur

8.57 R12.3 : Informations système

Code SQL :

```
SELECT CURRENT_DATE, CURRENT_TIMESTAMP;
```

Application : Date et heure du serveur

Retour : Utilisable dans les rapports datés

CATÉGORIE 13 : REQUÊTES MÉTIER AVANCÉES

Analyses décisionnelles pour la gestion de la cuisine.

8.58 R13.1 : Rentabilité menus

Code SQL :

```
SELECT
   M.NOM_MENU,
   M.PRIX_MENU,
   SUM(COALESCE(CMP.QUANTITE * I.PRIX_UNIT, 0)) AS COUT_TOTAL,
   M.PRIX_MENU - SUM(COALESCE(CMP.QUANTITE * I.PRIX_UNIT, 0)) AS MARGE
FROM MENU M
JOIN CONSTITUTION C ON M.CODE_MENU = C.CODE_MENU
JOIN RECETTE R ON C.CODE_REC = R.CODE_REC
LEFT JOIN COMPOSITION CMP ON R.CODE_REC = CMP.CODE_REC
LEFT JOIN INGREDIENT I ON CMP.CODE_ING = I.CODE_ING
GROUP BY M.CODE_MENU, M.NOM_MENU, M.PRIX_MENU
ORDER BY MARGE DESC;
```

Application : Analyse rentabilité par menu (tri marge décroissante)

8.59 R13.2 : Analyse ABC (80/20)

Code SQL :

```
WITH VALEURS AS (
   SELECT NOM_ING, (QTE_STOCK * PRIX_UNIT) AS VALEUR,
```

```
    SUM(QTE_STOCK * PRIX_UNIT) OVER () AS TOTAL
  FROM INGREDIENT
),
CLASSEMENT AS (
  SELECT NOM_ING, VALEUR,
    SUM(VALEUR) OVER (ORDER BY VALEUR DESC) AS CUMULE,
    ROUND(SUM(VALEUR) OVER (ORDER BY VALEUR DESC) / TOTAL * 100, 2) AS PCT
  FROM VALEURS
)
SELECT NOM_ING, VALEUR, PCT,
  CASE WHEN PCT <= 80 THEN 'A' WHEN PCT <= 95 THEN 'B' ELSE 'C' END AS CLASSE
FROM CLASSEMENT
ORDER BY VALEUR DESC;
```

Application : Classification ABC : A=80% valeur, B=15%, C=5%

8.60 R13.3 : Matrice compétences

Code SQL :

```
SELECT
  C.NOM_CHEF,
  COUNT(CASE WHEN R.CAT_REC = 'Entrée' THEN 1 END) AS Entrees,
  COUNT(CASE WHEN R.CAT_REC = 'Plat' THEN 1 END) AS Plats,
  COUNT(CASE WHEN R.CAT_REC = 'Dessert' THEN 1 END) AS Desserts,
  COUNT(*) AS Total
FROM CHEF C
JOIN REALISATION RE ON C.CODE_CHEF = RE.CODE_CHEF
JOIN RECETTE R ON RE.CODE_REC = R.CODE_REC
GROUP BY C.CODE_CHEF, C.NOM_CHEF;
```

Application : Tableau de bord polyvalence des chefs

8.61 R13.4 : Recettes réalisables

Code SQL :

```
SELECT R.NOM_REC
FROM RECETTE R
WHERE NOT EXISTS (
  SELECT 1 FROM COMPOSITION C
  JOIN INGREDIENT I ON C.CODE_ING = I.CODE_ING
  WHERE C.CODE_REC = R.CODE_REC AND C.QUANTITE > I.QTE_STOCK
);
```

Application : Plats préparables immédiatement (stock suffisant)

8.62 R13.5 : Tendances RH

Code SQL :

```
SELECT
   DATE_TRUNC('month', DATE_EMB) AS Mois,
   COUNT(*) AS Recrutements
FROM CHEF
GROUP BY DATE_TRUNC('month', DATE_EMB)
ORDER BY Mois;
```

Application : Historique des embauches par mois

9. CONCLUSION

Ce document a présenté la conception et la réalisation d'une base de données relationnelle pour la gestion d'une cuisine professionnelle. À travers les différents chapitres, nous avons suivi une démarche méthodologique rigoureuse fondée sur les principes de l'algèbre relationnelle et de la normalisation.

Synthèse des résultats

Notre travail a abouti à la création d'une base de données complète comprenant :

- Cinq entités principales : INGREDIENT, RECETTE, MENU, CHEF et FOURNISSEUR
- Trois tables d'association : COMPOSITION, CONSTITUTION et REALISATION
- Un schéma entièrement normalisé en 3FN/BCNF
- Le code SQL complet de création et d'interrogation

La modélisation conceptuelle réalisée avec la méthode Merise a permis de capturer fidèlement les besoins métier du domaine culinaire. La matrice des dépendances fonctionnelles a été un outil précieux pour identifier les liens entre les données et construire un modèle cohérent.

Apports de l'algèbre relationnelle

L'utilisation de l'algèbre relationnelle nous a permis de :

- Formuler précisément les requêtes utilisateurs de manière indépendante de toute implémentation

- Comprendre les fondements théoriques du langage SQL
- Optimiser les requêtes en choisissant l'ordre approprié des opérations
- Maîtriser les opérations de jointure essentielles pour l'exploitation d'une base de données relationnelle

Importance de la normalisation

Le processus de normalisation a été crucial pour garantir la qualité de notre base de données. En atteignant la 3FN (et même la BCNF), nous avons assuré :

- L'absence de redondances inutiles de données
- La prévention des anomalies de mise à jour
- L'intégrité et la cohérence des informations
- Une maintenance facilitée du système

Perspectives d'évolution

Notre base de données pourrait être enrichie et étendue dans plusieurs directions :

- Ajout de fonctionnalités de gestion des commandes fournisseurs avec historique
- Intégration de la gestion des coûts et de la comptabilité analytique
- Développement d'un module de planification des productions
- Création d'interfaces utilisateur pour faciliter l'exploitation
- Implémentation de mécanismes de sécurité et de contrôle d'accès

Cette étude nous a permis de mettre en pratique les concepts théoriques fondamentaux des bases de données relationnelles. La maîtrise de ces concepts est

essentielle pour tout ingénieur en informatique, car les bases de données constituent le socle de la plupart des systèmes d'information modernes.

10. BIBLIOGRAPHIE

Ouvrages de référence :

- Codd E.F., « A Relational Model of Data for Large Shared Data Banks », Communications of the ACM, 1970
- Clouse Michelle, « Algèbre relationnelle - Guide pratique de conception d'une base de données relationnelle normalisée », Editions ENI, 2008
- Date C.J., « An Introduction to Database Systems », Addison-Wesley, 8ème édition, 2004
- Elmasri R., Navathe S., « Fundamentals of Database Systems », Pearson, 7ème édition, 2016
- Gardarin G., « Bases de données - Les systèmes et leurs langages », Eyrolles, 2003
- Tardieu H., Rochfeld A., Colletti R., « La méthode MERISE - Principes et outils », Editions d'Organisation, 1983

Ressources en ligne :

- Documentation PostgreSQL : https://www.postgresql.org/docs/
- Documentation MySQL : https://dev.mysql.com/doc/
- W3Schools SQL Tutorial : https://www.w3schools.com/sql/

11. GLOSSAIRE

Algèbre relationnelle : Langage de manipulation de données fondé sur la théorie des ensembles, défini par E.F. Codd, permettant de manipuler des relations pour produire de nouvelles relations.

Association : Lien sémantique entre plusieurs entités dans un modèle conceptuel de données.

Attribut : Caractéristique d'une entité ou d'une relation, correspondant à une colonne dans une table.

Base de données : Ensemble structuré et organisé de données stockées sur un support informatique.

Cardinalité : Nombre minimum et maximum d'occurrences d'une entité pouvant participer à une association.

Clé étrangère : Attribut (ou ensemble d'attributs) qui référence la clé primaire d'une autre table.

Clé primaire : Attribut (ou ensemble d'attributs) qui identifie de manière unique chaque tuple d'une relation.

Dépendance fonctionnelle : Relation entre deux attributs où la connaissance de la valeur de l'un détermine la valeur de l'autre.

Entité : Objet matériel ou immatériel du monde réel, caractérisé par des propriétés.

Extension : Contenu d'une relation à un instant donné (ensemble des tuples).

Forme normale : Niveau de normalisation d'une relation défini par des critères précis (1FN, 2FN, 3FN, BCNF).

Jointure : Opération de l'algèbre relationnelle combinant deux relations selon une condition.

Matrice des dépendances fonctionnelles : Outil permettant de visualiser et d'analyser les dépendances entre attributs.

MCD (Modèle Conceptuel de Données) : Modèle représentant les entités, leurs attributs et les associations entre entités.

Merise :Méthode française de conception de systèmes d'information.

MLD (Modèle Logique de Données) : Modèle dérivé du MCD précisant la structure des tables relationnelles.

Normalisation : Processus de structuration d'une base de données visant à éliminer les redondances et les anomalies.

Projection : Opérateur de l'algèbre relationnelle sélectionnant certains attributs d'une relation.

Relation : Sous-ensemble du produit cartésien de plusieurs domaines, représentée comme une table à deux dimensions.

Sélection : Opérateur de l'algèbre relationnelle filtrant les tuples selon une condition.

Schéma : Structure définissant les attributs d'une relation avec leurs domaines.

SGBD : Système de Gestion de Base de Données, logiciel permettant de créer, manipuler et administrer des bases de données.

SQL : Structured Query Language, langage standard de manipulation des bases de données relationnelles.

Tuple : Ligne d'une relation représentant une occurrence de l'entité modélisée.

CONCEPTION DE BASE DE DONNÉES

Application au domaine de la cuisine

Élève Ingénieur en Informatique

© 2025-2026 – IAI CAMEROON